

SQL Analytics II

Módulo IX



Temario

- Tips de Performance
 - Analizar
 - SELECT *
 - JOINS vs SubQuery
 - EXISTS vs IN / NOT IN
 - LIKE
 - WHERE
 - Búsqueda negativa (<> o !=)
 - Cuando correr consultas complejas

SQL

Tips de Performance

Analizar

Analizar:

- Debemos entender que información nos están solicitando, de no ser así seguirán enviando solicitudes de información. Por lo tanto, antes de escribir consultas SQL, pregunte lo siguiente:
 - ¿Quién necesita los datos?
 - ¿Qué información quieren?
 - ¿Qué nivel de detalle requieren?

Luego de definir estas incógnitas y verificar la información con las partes interesadas, podemos explorar "cómo escribir consultas" de manera más eficiente.

No usar * en SELECT:

- Si usamos * nos devuelve todas las columnas de la tabla o tablas en particular y lleva tiempo, por lo que es mejor usar el nombre de la columna que se requiere para el resultado.

SELECT *
FROM Sales.SalesOrderDetail AS sod

SELECT
sod.SalesOrderID,
sod.ProductID,
sod.LineTotal
FROM Sales.SalesOrderDetail AS sod



JOIN vs SubQuery:

- Es mejor usar JOIN en lugar de SubQuery ya que el JOIN, ya sea LEFT, RIGHT, INNER o OUTER solo verificó los registros que coinciden con los criterios.
- Mientras que en la subconsulta verifican todos los registros y luego devuelven el resultado.
- Por lo tanto el JOIN es mas eficiente ya que maneja una menor cantidad de datos.

SELECT

```
p.FirstName,  
p.LastName,  
(SELECT hre.JobTitle  
FROM HumanResources.Employee AS hre  
WHERE p.BusinessEntityID = hre.BusinessEntityID) AS JobTitle  
FROM Person.Person AS p
```

SELECT

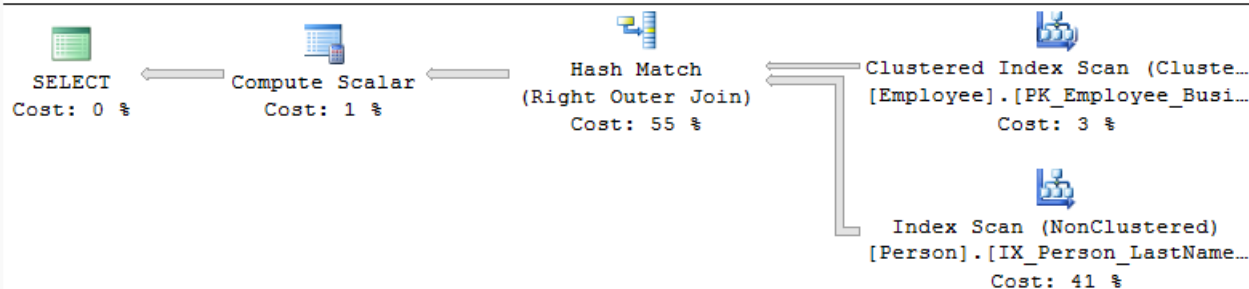
```
p.FirstName,  
p.LastName,  
hre.JobTitle  
FROM Person.Person AS p  
JOIN HumanResources.Employee AS hre  
ON p.BusinessEntityID = hre.BusinessEntityID
```

SELECT

```
p.FirstName,  
p.LastName,  
(SELECT hre.JobTitle  
FROM HumanResources.Employee AS hre  
WHERE p.BusinessEntityID = hre.BusinessEntityID) AS JobTitle  
FROM Person.Person AS p
```

Query 2: Query cost (relative to the batch): 53%

SELECT p.FirstName, p.LastName, (SELECT hre.JobTitle FROM HumanResources.Employee AS...



SQL

Tips de Performance

JOIN vs SubQuery

SELECT

p.FirstName,
p.LastName,
hre.JobTitle

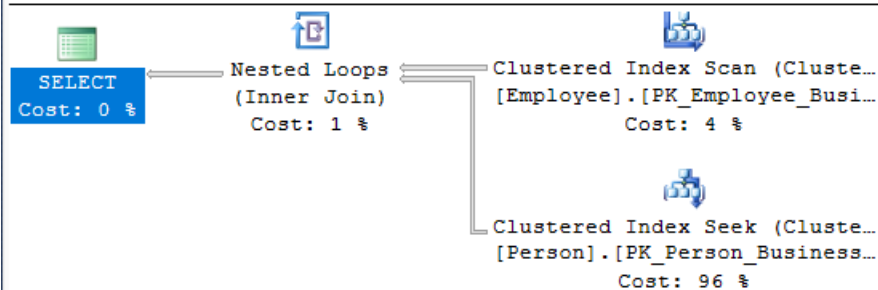
FROM Person.Person AS p

JOIN HumanResources.Employee AS hre

ON p.BusinessEntityID = hre.BusinessEntityID

Query 1: Query cost (relative to the batch): 47%

SELECT p.FirstName, p.LastName, hre.JobTitle FROM Person.Person AS p JOIN HumanResou...



SELECT

```
p.FirstName,  
p.LastName,  
(SELECT hre.JobTitle  
FROM HumanResources.Employee AS hre  
WHERE p.BusinessEntityID = hre.BusinessEntityID) AS JobTitle  
FROM Person.Person AS p
```

SELECT

```
p.FirstName,  
p.LastName,  
hre.JobTitle  
FROM Person.Person AS p  
JOIN HumanResources.Employee AS hre  
ON p.BusinessEntityID = hre.BusinessEntityID
```



EXISTS vs IN/NOT IN:

- Hay ocasiones en donde es necesario usar una subconsulta, en ese momento, utilice primero la función EXIST() si la subconsulta devuelva una gran cantidad de datos.
- Cuando los valores son finitos y conocidos es aconsejable la utilización del IN/NOT IN.
- La diferencia radica en que la función EXIST() nos devuelve un valor booleano (True or False) por lo que se procesara mas rápido que la función IN/NOT IN en la cual se compara cada valor de cada consulta.

```
SELECT ProductID, SUM(OrderQty*UnitPrice) AS Total
FROM Sales.SalesOrderDetail AS soh
WHERE EXISTS ( SELECT 1
                FROM Production.Product AS p
                WHERE Color = 'Black' AND
                      soh.ProductID = p.ProductID)
GROUP BY ProductID
```



```
SELECT ProductID, SUM(OrderQty*UnitPrice) AS Total
FROM Sales.SalesOrderDetail AS soh
WHERE soh.ProductID IN (SELECT p.ProductID
                        FROM Production.Product AS p
                        WHERE Color = 'Black' )
GROUP BY ProductID
```

LIKE:

- Para buscar una cadena, podemos usar el operador LIKE o el operador "=".
- La principal diferencia entre el operador LIKE y el operador "=" es que con el operador LIKE podemos buscar cadenas parciales usando comodines.
- En contraste, con el operador "=", buscamos coincidencias exactas.
- Si realizamos una búsqueda exacta, siempre es mejor usar el operador "=" porque utiliza columnas indexadas y es más rápido.

SQL

Tips de Performance

WHERE

WHERE:

- Filtrar los datos para incluir solo los datos que se necesitan puede mejorar drásticamente la velocidad de las consultas.
- La forma en que se haga dependerá completamente del problema que estemos tratando de resolver.
- Por ejemplo, si tiene datos de series temporales, limitarse a una ventana de tiempo pequeña puede hacer que las consultas se ejecuten mucho más rápidamente.

Búsqueda negativa (<> o !=):

- Muchas veces utilizamos la búsqueda negativa en condiciones como no igual (<>), distinto (!=) , etc.
- Debemos evitar ese tipo de búsqueda porque si damos los criterios de búsqueda exactos, al ejecutar la consulta si se obtiene ese tipo de datos, inmediatamente la consulta devuelve el resultado.
- Las búsquedas negativas disminuyen la velocidad de ejecución de consultas, por lo que debemos evitarlo.

Consultas complejas:

- Cuando tenga una consulta compleja y no haya forma de simplificarla, intente ejecutarlas en horarios donde el servidor no sea altamente usado.
- Ejecutar una consulta durante la noche es mejor cuando las consultas tienen
 - JOINS,
 - LIKES con WILDCARDS
 - SELECT DISTINCT
 - CROSS JOINS
 - Cálculos Complejos

Resumen Módulo IX

- Tips de Performance
 - Analizar
 - SELECT *
 - JOINS vs SubQuery
 - EXISTS vs IN / NOT IN
 - LIKE
 - WHERE
 - Búsqueda negativa (<> o !=)
 - Cuando correr la consulta